

1. Introdução e Metodologia

Na arquitetura **O-RAN (Open Radio Access Network)**, as xApps (aplicações do Near-RT RIC) operam de forma isolada e simultânea para gerenciar os ns de rádio. O problema intrínseco dessa arquitetura é o **Conflito de Controle**. O que acontece se uma QoS-xApp decide aumentar a potência e os recursos de rádio de uma célula simultaneamente a uma Energy-Savings-xApp que decide diminuí-los? A antena receberá requisitos contraditórios (RIC_CONTROL_REQUEST), resultando em oscilação agressiva (*ping-pong effect*) e degradação de SLAs.

A **xApp RDL (Resource and Decision Layer)** surge como uma camada de Orquestração Cognitiva (via RDP) que se posiciona de forma agnóstica como um árbitro entre o Near-RT RIC e as demais xApps. A metodologia adota **Domain-Driven Design (DDD)** para separar a lógica de decisão da infraestrutura de telecomunicações.

Na RDL:

1. As xApps parceiras não ativam comandos, apenas disparam intenções (RDL_ACTION_PROPOSAL) na rede RMR.
 2. A **RDL intercepta as propostas e as agrupa em uma Janela de Decisão (Decision Window de 200ms)**, abandonando o modelo reativo de primeiro a chegar (First-Come-First-Served).
 3. A IA funde os pedidos das xApps com a telemetria em tempo real (KPM) e **avalia o espaço combinatório das ações** para detectar oportunidades de complementaridade (executar múltiplas ações simultaneamente se aumentarem a utilidade global).
 4. A RDL decide utilizando **fórmulas de Acordo de Nível de Serviço (SLA) como TVS e EEVS ou Inteligência Artificial (MARL)**, valida regras físicas rígidas (Safety Guard) e despacha o comando final oficial (E2SM-RC).
-

2. Arquitetura Geral

A RDL atua como um Man-in-the-middle inteligente em **Ciclo Fechado**:

1. **Coleta (E2):** O E2NodeDiscoveryService localiza as antenas. O SubscriptionManager assina as métricas (E2SM-KPM). O payload ASN.1 APER é decodificado (e2ap_decoder e kpm_decoder).
2. **Proposta e Agrupamento Temporal:** Interceptação de RDL_ACTION_PROPOSAL da malha RMR, acumulando-as em um buffer até o fechamento da janela de 200ms.
3. **Arbitragem (Domínio/Agentes):** O PerceptionAgent gera o grafo situacional do lote inteiro. O ReasoningAgent escolhe a melhor resolução iterando as combinações e avaliando a utilidade com base em políticas rigorosas de SLA (como **TVS - Throughput Violation-based Selection** e **EEVS - EE Violation-based Selection**), Histórico ou IA (MAPPO).
4. **Guarda de Segurança:** O RefinementAgent valida restrições (limite percentual de blocos físicos, frequência de controle).

5. **Atualização (E2/RMR):** O comando `RC` formatado pelo `rc_encoder` e atirado `Radio` base pelo `ControlDispatcher`. O ID da decisão `RC` salvo no banco de dados distribuído (SDL) via `sdl_repository` para esperar a confirmação (ACK).
-

3. Estrutura do Projeto e Componentes

A estrutura obedece aos padrões Clean Architecture:

- `configs/`: Schema de configuração (`xapp_descriptor.json`, `schema.json`, `routes.rt.template`).
 - `deploy/kubernetes/`: Manifestos O-RAN compliant para a xApp (Deployment, Service).
 - `docker/`: Dockerfile Multi-stage build com usuário restrito `xapp`.
 - `scripts/`: Injeção de variáveis RMR e coleta automática de evidências de experimentação.
 - `src/agents/`: Motores de percepção (Grafos), inteligência artificial MARL (MAPPO) e *Safety Guards*.
 - `src/coordination/`: Despachantes (`control_dispatcher`) para gerir o handshake O-RAN (Request, Ack, Failure).
 - `src/domain/`: dataclasses restritas que garantem a integridade das Entidades (Proposals, Decisions, Conflicts).
 - `src/e2/`: Decodificadores e codificadores de carga útil específica E2AP, E2SM-KPM e E2SM-RC.
 - `src/infrastructure/`: Portas e Adaptadores, gerindo comunicação com SDL (Redis), Subscription Manager e API E2 Manager.
 - `src/observability/`: Métricas precisas em Prometheus (`rdl_kpm_indications_total`), servidor Uvicorn de Health e Logging Estruturado (JSON).
-

4. O Schema (RDL Action Proposal)

Para que xApps parceiras comuniquem-se com a RDL, o modelo JSON estrito (protocolado no Domínio) exige:

```
{
  "schema_version": "1.0",
  "proposal_id": "uuid",
  "source_xapp": "qos_app_1",
  "timestamp": "2026-08-05T12:00:00Z",
  "valid_until": "2026-08-05T12:00:01Z",
  "target": {
    "node_id": "gnb_1",
    "cell_id": "cell_alpha",
    "ue_ids": [],
    "slice_ids": []
  },
  "action": {
    "type": "PRB_ALLOCATION",
    "parameters": {"prb_value": 40}
  }
}
```

```
"priority": 100
}
```

5. Modelo de Funcionamento da Inteligência (MARL)

A espinha dorsal cognitiva (`agents/marl/`) utiliza **Multi-Agent Proximal Policy Optimization (MAPPO)**. Quando ocorre um Conflito Indireto (xApps controlando parâmetros diferentes que destroem o mesmo SLA em cadeia), o Reasoning Agent descarta regras estáticas (Prioridade) e aciona a rede neural via PyTorch. A IA aprende, iterativamente, as consequências não-lineares da Rádio Frequência para recomendar a intenção que mantém o Equilíbrio de Nash.

6. Conclusão

A reconstrução arquitetural sob o formato "Zero to Hero" garantiu que o projeto RDL evoluísse de uma pesquisa monolítica (Proof-of-Concept em laboratório) para uma xApp determinística, distribuída, e escalável (Production-Ready). Através da isolamento do domínio cognitivo em detrimento da casca de comunicação E2, o empacotamento Multi-stage Non-root nativamente endossado pelas especificações OSC Near-RT RIC foi alcançado sem comprometer o núcleo matemático de Inteligência Artificial para resolução de conflitos na RAN 5G e 6G.